Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №2
на тему

СИММЕТРИЧНАЯ КИПТОГРАФИЯ.
СТБ 34.101.31-2011

БГУИР КП 6-05-0612-02 016 ПЗ

Выполнил студент гр. 353502
МАКСИМЕНКОВ
Владислав Александрович

(дата, подпись студента)

Проверил ассистент каф.Информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 6
Приложение А (обязательное) Листинг программного кода 7

ВВЕДЕНИЕ

В данной лабораторной работе рассматривается симметричная криптография в соответствии с требованиями СТБ 34.101.31-2011. Настоящий стандарт определяет семейство криптографических алгоритмов, предназначенных для обеспечения конфиденциальности и контроля целостности двоичных данных. В стандарте рассматриваются различные группы алгоритмов, в том числе алгоритмы шифрования в режиме простой замены, сцепления блоков, гаммирования с обратной связью и счетчика, а также алгоритмы выработки имитовставки и хеширования. В рамках данной лабораторной работы требуется реализовать программные средства шифрования и дешифрования текстовых файлов в режимах простой замены и гаммирования с обратной связью.

В качестве базового алгоритма в программном средстве реализован блочный алгоритм шифрования BELT, предусмотренный СТБ 34.101.31-2011. Обработка данных выполняется блоками фиксированной длины, а для выполнения криптографического преобразования используется 256-битный ключ. Реализованное программное средство обеспечивает обработку текстовых файлов, выполнение операций зашифрования и расшифрования, а также работу с файлами произвольной длины в режиме гаммирования с обратной связью.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для шифрования и дешифрования текстовых файлов с использованием алгоритма BELT в соответствии с СТБ 34.101.31-2011. Реализация выполнена в операционной системе Ubuntu 24.04 LTS с использованием стандартных средств языка C++ и системы сборки CMake.

Программа организована модульно. В главном модуле выполняется разбор аргументов командной строки, чтение и запись файлов, выбор режима работы и запуск соответствующей процедуры шифрования или расшифрования. В отдельном модуле реализованы основные преобразования алгоритма BELT, включая операции загрузки и сохранения 32-битных слов, циклического сдвига, нелинейного преобразования, формирования подключей и непосредственного зашифрования и расшифрования блока.

Основная процедура `beltEncrypt` выполняет восемь итераций преобразования. В каждой итерации используются соответствующие 32-битные части ключа, выполняются сложение по модулю 2^{32}, нелинейное преобразование `G`, циклические сдвиги и операции XOR. После завершения всех итераций из полученных 32-битных слов формируется 128-битный выходной блок.

Процедура `beltDecrypt` выполняет обратное преобразование блока. Для этого используется обратная последовательность подключей и обратная последовательность операций. Таким образом, применение расшифрования к результату зашифрования при использовании того же ключа позволяет восстановить исходный блок.

Для режима `ECB` перед зашифрованием выполняется дополнение исходного файла. В программе используется дополнение, при котором количество добавляемых байтов определяется размером блока, а каждый добавленный байт содержит значение длины дополнения. Это позволяет представить файл произвольной длины как последовательность полных 128-битных блоков. После расшифрования выполняется проверка и удаление дополнения. Если структура дополнения некорректна, программа сообщает об ошибке расшифрования.

В режиме гаммирования с обратной связью используется 128-битная синхропосылка. В соответствии с методическими материалами входное сообщение разбивается на блоки длиной 128 бит, причем последний блок может иметь длину менее 128 бит. При зашифровании очередной блок сообщения складывается по операции XOR с результатом преобразования предыдущего значения регистра. Начальное состояние регистра устанавливается равным синхропосылке.

В реализованной процедуре `cipherFeedback` начальным значением регистра обратной связи является фиксированная синхропосылка. Для каждого участка данных вычисляется значение гаммы путем зашифрования текущего состояния регистра. Затем гамма складывается с исходными данными посредством операции XOR. Полученный результат записывается в выходной файл.

При зашифровании после обработки очередного блока результат становится новым значением регистра обратной связи. При расшифровании в качестве следующего состояния используется соответствующий предыдущий блок шифртекста. Поэтому обработка каждого следующего блока зависит от результата предыдущего преобразования. Такой принцип соответствует алгоритму гаммирования с обратной связью, приведенному в СТБ 34.101.31-2011.

Особенностью реализованного режима `CFB` является возможность обработки последнего неполного блока. Для него фактический размер обрабатываемого участка определяется как минимум между размером блока и количеством оставшихся байтов. Поэтому при использовании данного режима дополнительное заполнение последнего блока не требуется.

Для запуска программы предусмотрены параметры командной строки, задающие режим работы, операцию, входной и выходной файлы. Поддерживаются режимы `ECB` и `CFB`, а для каждого режима доступны операции `enc` и `dec`. Пример вызова программы имеет следующий вид:

```
program ECB enc secret.txt secret.enc
```

где `ECB` определяет режим простой замены, `enc` — операцию зашифрования, `secret.txt` — исходный файл, а `secret.enc` — файл с результатом обработки.

Реализованные функции и классы приведены в Приложении А (листинг программного кода).

Для проверки работоспособности программного средства был выбран текстовый файл, содержащий обычный читаемый текст. Тестирование выполнялось отдельно для режимов простой замены и гаммирования с обратной связью. Для каждого режима сначала выполнялось зашифрование исходного файла, после чего полученный файл расшифровывался с использованием тех же ключа и синхропосылки. После расшифрования полученное содержимое сравнивалось с исходным файлом.

Рисунок 1.1 – Вывод программы при шифровании и расшифровании файла с использованием СТБ 34.101.31-2011

В результате проведенного тестирования в обоих реализованных режимах после расшифрования содержимое файла восстанавливается и соответствует исходным данным. Для режима `ECB` дополнительно проверяется корректность удаления дополнения после расшифрования. Для режима `CFB` проверяется обработка как полных, так и неполных блоков исходного файла.

Таким образом, проведенные испытания позволяют проверить корректность реализации базового блочного преобразования, а также правильность функционирования режимов простой замены и гаммирования с обратной связью.

ВЫВОД

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для шифрования и дешифрования текстовых файлов с использованием алгоритма BELT в соответствии с СТБ 34.101.31-2011. Были реализованы два предусмотренных заданием режима: простой замены и гаммирования с обратной связью.

В реализации предусмотрена обработка 128-битных блоков данных и 256-битного ключа, выполнение базовых криптографических преобразований, зашифрование и расшифрование блоков, а также чтение и запись файлов. Для режима простой замены реализовано дополнение последнего блока, а для режима гаммирования с обратной связью обеспечена возможность обработки сообщения произвольной длины за счет обработки неполного последнего участка.

Проведенное тестирование показало, что после последовательного зашифрования и расшифрования исходное содержимое файла восстанавливается. Полученные результаты подтверждают работоспособность реализованных процедур и позволяют на практике ознакомиться с принципами симметричного блочного шифрования и использованием алгоритма BELT в различных режимах работы.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – main.cpp

```
#include <iostream>
#include <algorithm>
#include <string>
#include "belt.h"
#include "utils.h"

const std::string FIXED_KEY_HEX =
"0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF";
const std::string FIXED_IV_HEX = "00112233445566778899AABBCCDDEEFF";

static Key getFixedKey() {
    Key key{};
    hexToBytes(FIXED_KEY_HEX, key.data(), key.size());
    return key;
}

static bool simpleReplacement(bool decrypt, const std::string& inputFile,
const std::string& outputFile) {
    Key key = getFixedKey();
    std::vector<Byte> data;

    if (!readFile(inputFile, data)) {
        std::cerr << "Error: could not open input file '" << inputFile <<
"'\n";
        return false;
    }

    if (!decrypt) { // encrypt
        addPadding(data);
        std::vector<Byte> encrypted(data.size());

        for (size_t pos = 0; pos < data.size(); pos += BLOCK_SIZE) {
            Block inputBlock{};
            std::copy_n(data.begin() + pos, BLOCK_SIZE, inputBlock.begin());
            Block outputBlock = beltEncrypt(inputBlock, key);
            std::copy(outputBlock.begin(), outputBlock.end(),
encrypted.begin() + pos);
        }

        if (!writeFile(outputFile, encrypted)) {
            std::cerr << "Error: could not write to file '" << outputFile <<
"'\n";
            return false;
        }
        std::cout << "[OK] Encryption (ECB) completed successfully.\n";
    } else { // decrypt
        if (data.empty() || data.size() % BLOCK_SIZE != 0) {
            std::cerr << "Error: ciphertext size must be a multiple of 16
bytes.\n";
            return false;
        }

        std::vector<Byte> decrypted(data.size());

        for (size_t pos = 0; pos < data.size(); pos += BLOCK_SIZE) {
            Block inputBlock{};
```

```
            std::copy_n(data.begin() + pos, BLOCK_SIZE, inputBlock.begin());
            Block outputBlock = beltDecrypt(inputBlock, key);
            std::copy(outputBlock.begin(), outputBlock.end(),
decrypted.begin() + pos);
        }

        if (!removePadding(decrypted)) {
            std::cerr << "Decryption error: invalid key or corrupted data
(padding error).\n";
            return false;
        }

        if (!writeFile(outputFile, decrypted)) {
            std::cerr << "Error: could not write to file '" << outputFile <<
"'\n";
            return false;
        }
        std::cout << "[OK] Decryption (ECB) completed successfully.\n";
    }
    return true;
}

static bool cipherFeedback(bool decrypt, const std::string& inputFile, const
std::string& outputFile) {
    Key key = getFixedKey();
    Block iv = getFixedIV();
    std::vector<Byte> data;

    if (!readFile(inputFile, data)) {
        std::cerr << "Error: could not open input file '" << inputFile <<
"'\n";
        return false;
    }

    std::vector<Byte> result(data.size());
    Block previous = iv;

    for (size_t pos = 0; pos < data.size(); pos += BLOCK_SIZE) {
        Block gamma = beltEncrypt(previous, key);
        size_t currentSize = std::min(BLOCK_SIZE, data.size() - pos);
        Block currentBlock{};

        for (size_t j = 0; j < currentSize; ++j) {
            currentBlock[j] = data[pos + j] ^ gamma[j];
            result[pos + j] = currentBlock[j];
        }

        if (!decrypt) {
            for (size_t j = 0; j < currentSize; ++j) {
                previous[j] = result[pos + j];
            }
        } else {
            for (size_t j = 0; j < currentSize; ++j) {
                previous[j] = data[pos + j];
            }
        }
    }

    if (!writeFile(outputFile, result)) {
        std::cerr << "Error: could not write to file '" << outputFile <<
"'\n";
        return false;
```

```
    }

    std::cout << "[OK] " << (decrypt ? "Decryption" : "Encryption")
              << " (CFB) completed successfully.\n";
    return true;
}


void printUsage(const char* programName) {
    std::cerr << "Usage: " << programName << " <mode: ECB|CFB> <enc|dec>
<input> <output>\n"
              << "Parameters:\n"
              << "  mode   : ECB (Electronic Codebook) or CFB (Cipher
Feedback)\n"
              << "  action : enc (Encrypt) or dec (Decrypt)\n"
              << "  input  : Path to the input file\n"
              << "  output : Path to the output file\n\n"
              << "Example: " << programName << " ECB enc secret.txt
secret.enc\n";
}


int main(int argc, char* argv[]) {
    if (argc != 5) {
        printUsage(argv[0]);
        return 1;
    }

    // args
    std::string modeStr = argv[1];
    std::string opStr   = argv[2];
    std::string inFile  = argv[3];
    std::string outFile = argv[4];

    if (modeStr != "ECB" && modeStr != "CFB") {
        std::cerr << "Error: unknown mode '" << modeStr << "'. Valid modes:
ECB, CFB.\n";
        return 1;
    }

    if (opStr != "enc" && opStr != "dec") {
        std::cerr << "Error: unknown action '" << opStr << "'. Valid actions:
enc, dec.\n";
        return 1;
    }

    bool decrypt = (opStr == "dec");
    bool success = false;

    // run
    if (modeStr == "ECB") {
        success = simpleReplacement(decrypt, inFile, outFile);
    } else if (modeStr == "CFB") {
        success = cipherFeedback(decrypt, inFile, outFile);
    }

    return success ? 0 : 1;
}
```

Листинг A.2 – belt.cpp

```
#include "belt.h"

namespace {
    uint32_t load32(const Byte* p) {
        return static_cast<uint32_t>(p[0]) | (static_cast<uint32_t>(p[1]) <<
8) |
               (static_cast<uint32_t>(p[2]) << 16) |
(static_cast<uint32_t>(p[3]) << 24);
    }

    void store32(Byte* p, uint32_t v) {
        p[0] = static_cast<Byte>(v);
        p[1] = static_cast<Byte>(v >> 8);
        p[2] = static_cast<Byte>(v >> 16);
        p[3] = static_cast<Byte>(v >> 24);
    }

    uint32_t rotl32(uint32_t x, unsigned int r) {
        return (x << r) | (x >> (32 - r));
    }

    uint32_t G(uint32_t x, unsigned int r) {
        uint32_t y = (static_cast<uint32_t>(H[(x >> 24) & 0xFF]) << 24) |
                     (static_cast<uint32_t>(H[(x >> 16) & 0xFF]) << 16) |
                     (static_cast<uint32_t>(H[(x >> 8)  & 0xFF]) << 8)  |
                      static_cast<uint32_t>(H[x & 0xFF]);
        return rotl32(y, r);
    }

    std::array<uint32_t, 8> keyWords(const Key& key) {
        std::array<uint32_t, 8> k{};
        for (int i = 0; i < 8; ++i) k[i] = load32(key.data() + 4 * i);
        return k;
    }

    const int KeyIndex[8][7] = {
        {0, 1, 2, 3, 4, 5, 6}, {7, 0, 1, 2, 3, 4, 5},
        {6, 7, 0, 1, 2, 3, 4}, {5, 6, 7, 0, 1, 2, 3},
        {4, 5, 6, 7, 0, 1, 2}, {3, 4, 5, 6, 7, 0, 1},
        {2, 3, 4, 5, 6, 7, 0}, {1, 2, 3, 4, 5, 6, 7}
    };
}


Block beltEncrypt(const Block& input, const Key& key) {
    const auto k = keyWords(key);
    uint32_t a = load32(input.data() + 0), b = load32(input.data() + 4);
    uint32_t c = load32(input.data() + 8), d = load32(input.data() + 12);
    uint32_t e, temp;

    for (int i = 0; i < 8; ++i) {
        b ^= G(a + k[KeyIndex[i][0]], 5);
        c ^= G(d + k[KeyIndex[i][1]], 21);
        a -= G(b + k[KeyIndex[i][2]], 13);
        e = G(b + c + k[KeyIndex[i][3]], 21) ^ static_cast<uint32_t>(i + 1);
        b += e; c -= e;
        d += G(c + k[KeyIndex[i][4]], 13);
        b ^= G(a + k[KeyIndex[i][5]], 21);
        c ^= G(d + k[KeyIndex[i][6]], 5);
        temp = a; a = b; b = temp;
        temp = c; c = d; d = temp;
        temp = b; b = c; c = temp;
```

```
    }

    Block output{};
    store32(output.data() + 0,  b);
    store32(output.data() + 4,  d);
    store32(output.data() + 8,  a);
    store32(output.data() + 12, c);
    return output;
}

Block beltDecrypt(const Block& input, const Key& key) {
    const auto k = keyWords(key);
    uint32_t a = load32(input.data() + 0), b = load32(input.data() + 4);
    uint32_t c = load32(input.data() + 8), d = load32(input.data() + 12);
    uint32_t e, temp;

    for (int i = 0; i < 8; ++i) {
        int r = 7 - i;
        b ^= G(a + k[KeyIndex[r][6]], 5);
        c ^= G(d + k[KeyIndex[r][5]], 21);
        a -= G(b + k[KeyIndex[r][4]], 13);
        e = G(b + c + k[KeyIndex[r][3]], 21) ^ static_cast<uint32_t>(r + 1);
        b += e; c -= e;
        d += G(c + k[KeyIndex[r][2]], 13);
        b ^= G(a + k[KeyIndex[r][1]], 21);
        c ^= G(d + k[KeyIndex[r][0]], 5);
        temp = a; a = b; b = temp;
        temp = c; c = d; d = temp;
        temp = a; a = d; d = temp;
    }

    Block output{};
    store32(output.data() + 0,  c);
    store32(output.data() + 4,  a);
    store32(output.data() + 8,  d);
    store32(output.data() + 12, b);
    return output;
}

void addPadding(std::vector<Byte>& data) {
    size_t padding = BLOCK_SIZE - (data.size() % BLOCK_SIZE);
    if (padding == 0) {
        padding = BLOCK_SIZE;
    }
    data.insert(data.end(), padding, static_cast<Byte>(padding));
}

bool removePadding(std::vector<Byte>& data) {
    if (data.empty() || data.size() % BLOCK_SIZE != 0) {
        return false;
    }
    Byte padding = data.back();
    if (padding == 0 || padding > BLOCK_SIZE || padding > data.size()) return
false;
    for (size_t i = 0; i < padding; ++i) {
        if (data[data.size() - 1 - i] != padding) {
            return false;
        }
    }
    data.resize(data.size() - padding);
    return true;
}
```